

**Федеральное государственное автономное образовательное
учреждение высшего образования
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Высшая школа бизнеса**

**Контрольная домашняя работа
по курсу «Базы данных»
«База данных библиотек г. Мичуринск»**

по направлению подготовки 38.03.05 Бизнес-информатика
образовательная программа «Бизнес-информатика»

Проект выполнила команда №37:
Гущина Александра Геннадьевна (ББИ 2104)
Кузнецов Владимир Петрович (ББИ 2104)
Лоик Екатерина Викторовна (ББИ 2104)
Новиков Иван Александрович (ББИ 2104)
Чекурашева Алиса Ильинична (ББИ 2101)

Москва 2022

Оглавление

1. Информация о командной работе

1.1. Роли в команде

2.1. Ссылка на видеоролик

2. Описание предметной области

2.1. Общие сведения о предметной области

2.2. Цель создания базы данных

2.3. Возможные пользователи базы данных

2.4. Возможные сценарии взаимодействия пользователей с базой данных

2.5. Список основных сущностей, их атрибутов и первичных ключей

2.6. Описание связей между сущностями

2.7. Алгоритмы обработки данных

2.8. Планируемый перечень отчётных форм

2.9. Возможная архитектура программного продукта на основе вашей базы данных

3. Проект базы данных

3.1. Инфологическая модель предметной области

3.2. Дatalogическая модель данных и типы данных

3.3. Нормализация базы данных

3.4. Заполнение таблиц данными

4. Разработка

4.1. Настройка индексов

4.2. Запросы

4.3. Хранимые процедуры

4.4. Триггер

4.5. Функции

4.6. Представления

5. Отчёты

5.1. Отчёт в Tableau

5.2. Отчёт в Tableau

5.3. Отчёт в Excel

1. Информация о командной работе

1.1. Роли в команде

1. Гущина Александра – создание видеоролика с презентацией выполненной работы, создание триггера
2. Кузнецов Владимир – настройка индексов, разработка концептуальной модели данных (даталогической модели), разработка диаграммы БД, создание хранимых процедур
3. Лоик Екатерина – разработка модели предметной области (инфологической модели), разработка отчёта в MS Excel 2016/2019, Power BI или в Tableau, создание представлений
4. Новиков Иван – размещение базы данных в СУБД, заполнение базы данных тестовыми данными, разработка полезных запросов (только SELECT)
5. Чекурашева Алиса – координация работы команды, анализ предметной области, написание отчета по проделанной работе, создание функций

2.1. Ссылка на видеоролик

https://youtu.be/cP_k541gTRE - Это видео о любви, предательстве и базах данных. Приятного просмотра! Обязательно смотреть начало и конец.

2. Описание предметной области

2.1. Общие сведения о предметной области

Россия в 2022 году занимает 7 место в рейтинге самых читающих стран. Граждане нашей страны проводят за книгами около 7 часов. Россияне регулярно попадают в топ-10. Для многих книги являются частым способом развлечения и саморазвития, поэтому несмотря на угасающий интерес к печатным изданиям, библиотеки по прежнему остаются точкой притяжения людей.

В малых городах открытый доступ к информации и книгам является более острой проблемой, а развитие библиотечной инфраструктуры тоже очень низкое (в отличие от крупных городов). Поэтому в качестве предметной области мы выбрали библиотеки небольшого города. В качестве примера мы взяли библиотеки г. Мичуринска. В нём проживает около 90,5 тысяч человек, почти 5% жителей являются читателями различных библиотек. В центральной библиотеке содержится более 30 тысяч печатных изданий и еще более 10 тысяч других носителей информации (например, диски, пленки, микрофильмы).

Поскольку предметная область выбрана достаточно обширно, то создается возможность масштабирования полученного решения: подобная база данных легко применима в любом небольшом городе, а также возможно с небольшими дополнениями использовать и в крупных городах.

2.2. Цель создания базы данных

Большое собрание книг упорядочивается картотеками, формуляры читателей ведутся в бумажном варианте и могут быть утеряны, что создает большую нагрузку на библиотекарей. Посетители библиотеки тоже сталкиваются с проблемами поиска необходимой книги среди множества библиотек. Для аналитиков данные помогут понять направления развития: например, какие издания и жанры более популярны или сколько читателей регулярно посещают здание библиотеки, кто является целевой аудиторией нынешних учреждений. Также хранение данных в таком формате ускоряет процессы, поскольку многие базы сейчас создаются в Excel таблицах или простейших неудобных сервисах.

На поддержание регулярной работы библиотек тратятся деньги из муниципальных бюджетов, поэтому так важно более качественно следить за работой учреждений. Современный формат библиотек становится устаревшим и неинтересным современной молодежи. Поэтому так важно получать и сохранять данные о существующем использовании библиотеки для дальнейшего развития подобного формата и сохранения учреждений.

2.3. Возможные пользователи базы данных

Базой данных библиотек малого города могут пользоваться:

- Сотрудники городской администрации
- Сотрудники Министерства культуры РФ, региона и города
- Читатели и посетители библиотек
- Сотрудники библиотек
- Сотрудники издательств
- SMM-отдел (те, кто ведут сайт и социальные сети библиотек города)
- Аналитики-исследователи

2.4. Возможные сценарии взаимодействия пользователей с базой данных

Существует множество сценариев взаимодействия с базой данных, но, выделяя ключевые для потенциальных пользователей, стоит показать следующие:

- Читатели и посетители заинтересованы в поиске литературы, может осуществляться по жанрам, названиям, библиотекам, какие книги доступны, а какие сейчас недоступны;
- Сотрудники гос. органов обычно ищут проблемы у существующих библиотек: есть ли нехватка книг, насколько актуальны библиотеки, какие сотрудники сейчас ответственные и работают, также интересуют общие отчётные формы;
- Для сотрудников библиотек и SMM-отдела важны данные, чтобы расположить в публичный доступ, возможность изменения базы данных, обработки и поиска существующих проблем конкретной библиотеки, поиск должников, обработка записей в формулярах и сводные таблицы;

- Аналитики-исследователи заинтересованы в доступе к открытым таблицам базы данных и общим сводным отчётам.

2.5. Список основных сущностей, их атрибутов и первичных ключей

Сущность: Books

Описание: В сущности хранятся данные о каждом печатном издании библиотек: название и автор, издание, а также отражено в какой библиотеке хранится и доступно ли оно сейчас для читателей.

Атрибуты: id_book, year, name, is_taken

Первичный ключ: id_book

Сущность: Libraries

Описание: Сущность содержит информацию обо всех библиотеках города, специализации и адресах.

Атрибуты: id_library, address, specification, name

Первичный ключ: id_library

Сущность: Workers

Описание: Сущность состоит из данных о сотрудниках всех библиотек.

Атрибуты: position, name, id_worker

Первичный ключ: id_worker

Сущность: Entries in the form

Описание: В сущности хранятся данные формуляров читателей: какая книга, когда взята и когда ее необходимо вернуть, а также сотрудник библиотеки, оформивший книгу, и читатель

Атрибуты: id_entry, date of receipt, return date

Первичный ключ: id_entry

Сущность: Readers

Описание: Сущность хранит данные о читателях, в частности контактные данные, по которым можно уточнить информацию или напомнить о задолженности

Атрибуты: id_reader, name, contact details

Первичный ключ: id_reader

Сущность: Publishers

Описание: В сущности содержатся данные об издательских домах и их контактной информации

Атрибуты: id_publisher, name, address, contact details

Первичный ключ: id_publisher

Сущность: Authors

Описание: Сущность состоит из данных об авторах книг.

Атрибуты: id_author, name

Первичный ключ: id_author

Сущность: Genres

Описание: Сущность содержит в себе названия жанров.

Атрибуты: id_genre, name

Первичный ключ: id_genre

2.6. Описание связей между сущностями

Libraries_Books

Связь один к одному. Каждая конкретная книга с конкретным ID лежит в конкретной библиотеке, даже если книги с конкретным издательством повторяются, им выдаются разные ID, поскольку физически это 2 разные книги.

Libraries_Workers

Связь один ко многим, поскольку сотрудники библиотек не повторяются (т.е. обычно один сотрудник не работает в нескольких библиотеках сразу). А в одной библиотеке работают много сотрудников.

Workers_Entries in form

Связь многие ко многим, поскольку может быть несколько итераций с одной записью в формуляре: один сотрудник может отдать книгу читателю, второй может продлить, а третий – принять. Соответственно 1 формуляру может соответствовать несколько ответственных сотрудников. И, соответственно, один сотрудник ответственен за записи во многих формулярах.

Entries in form Readers

Связь один ко многим. Записей для одного читателя в формуляре может быть много (поскольку читатели за всё время берут больше 1 книги:)). Однако 1 запись в формуляре соответствует только 1 конкретный читатель, который эти книги берет.

Entries in form Books

Связь многие ко многим. Поскольку за 1 раз можно взять несколько книг сразу и в разных формулярах может быть одна и та же книга, если ее берут после.

Books Authors

Связь многие ко многим, поскольку разные авторы могут писать одну и ту же книгу вместе, а потом порознь (например, знаменитые Ильф и Петров писали некоторые произведения поодиночке, но чаще вместе) или их произведения помещены в одно собрание. А также обычно автору принадлежит больше чем 1 произведение.

Books Publishers

Связь 1 ко многим. 1 конкретный экземпляр книги может напечатать только 1 издательский дом, однако 1 издательский дом публикует множество книг.

Books Genres

Связь 1 ко многим. 1 книге соответствует один основной жанр (для упрощения анализа интересов читателей), в одном жанре могут быть написаны множество книг.

2.7. Алгоритмы обработки данных

Есть два основных алгоритма обработки данных:

1. Линейный поиск значения в таблице. Он является самым простым и самым долгим по скорости обработки (его сложность — $O(n)$, где n — количество строк);
2. Бинарный поиск, который работает быстрее линейного (его сложность — $\log_2(n)$, где n — количество строк). Алгоритм заключается в том, что диапазон поиска все время сокращается в 2 раза.

2.8. Планируемый перечень отчётных форм

Для проекта библиотек нет отчётных форм по деньгам, однако важность представляют: состояние книжного фонда (его старость, количество, требуемые важные экземпляры, стоимость закупок новых книг), качество работы сотрудников библиотеки (количество записей в день и прочие показатели), общие показатели (количество книг, читателей (и их возраст), записей в день, сотрудников) и интерес людей к литературе (какие жанры и авторы популярны, какие есть изменения в интересах и предпочтениях, сколько посетителей приходит).

2.9. Возможная архитектура программного продукта на основе вашей базы данных

Из данной СУБД можно создать приложение для сотрудников библиотеки, посетителей и аналитиков. Удобнее всего реализовать его в веб-версии на сайте библиотек для “внешних” пользователей базы данных (т.е. тех, у кого нет доступа к изменению базы данных) и в десктоп-версии для сотрудников библиотеки и государственных органов.

На главной странице для внешних пользователей удобно расположить каталог книжного фонда всего города или конкретной библиотеки с возможностями фильтра и поиска по названию, авторам, жанрам, годам, издательствам и доступности. Также можно создать вкладку с личным формуляром, который бы выводил все записи, соответствующие ID читателя.

Для сотрудников на главной необходимо выводить информацию о нынешних должниках-читателя, возможность добавить новые записи в формуляр или изменить данные в формулярах, а в случае прихода нового посетителя – его регистрация и создание личного формуляра читателя. Также в отдельной вкладке доступ к изменению книжного каталога (с учетом уровня доступа! чтобы, например, стажеры случайно не испортили содержание базы данных).

В крайней вкладке можно вывести отчёты о каждой библиотеке, которые помогут аналитикам и гос. органам в принятии решений и создании выводов.

3. Проект базы данных

3.1. Инфологическая модель предметной области

Ниже представлена инфологическая модель библиотек г. Мичуринска (Рисунок 1).

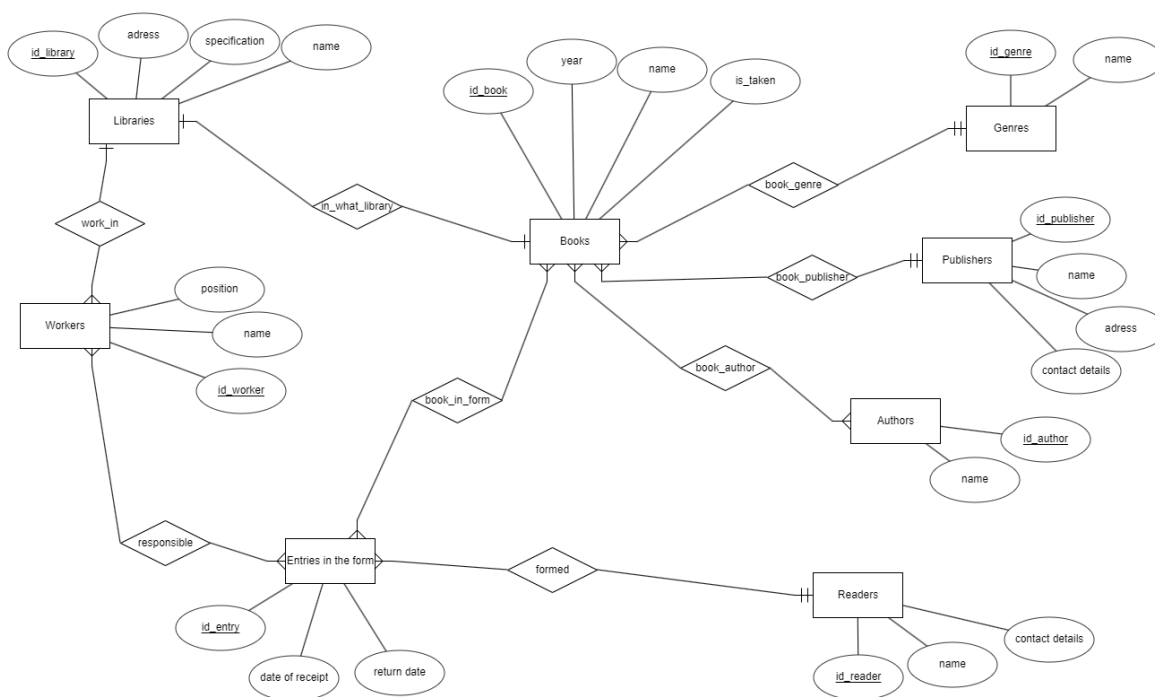


Рисунок 1. Инфологическая модель предметной области

3.2. Даталогическая модель данных и типы данных

На основе инфологической модели создана даталогическая, для реализации связей многие ко многим созданы 3 дополнительные таблицы: book_in_form, responsible, book_author (Рисунок 2). Также ниже есть диаграмма с типами данных в модели (Рисунок 3).

Все данные с состоянием not null, поскольку они критичны для работы системы, однако таблица Publishers не может быть всегда заполнена, поскольку некоторые издательства закрываются, а книги их издательств остаются (например, книги, выпущенные в СССР). А также у некоторых книг издатель может быть неизвестен. Поэтому в таблице Books издатель может быть null, в таблице издателей всё может быть null, кроме id_publisher и name.

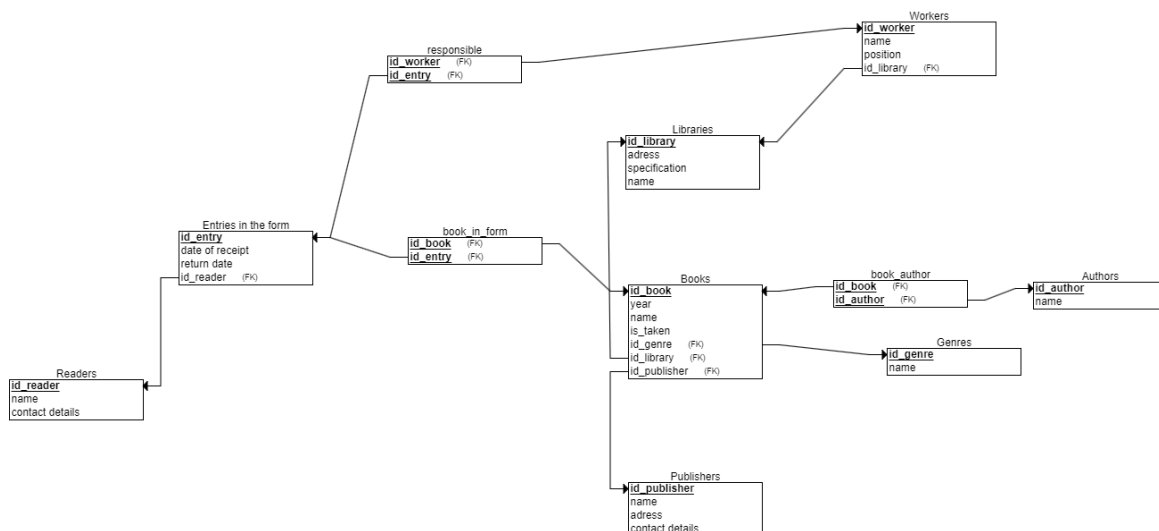


Рисунок 2. Дatalogическая модель данных

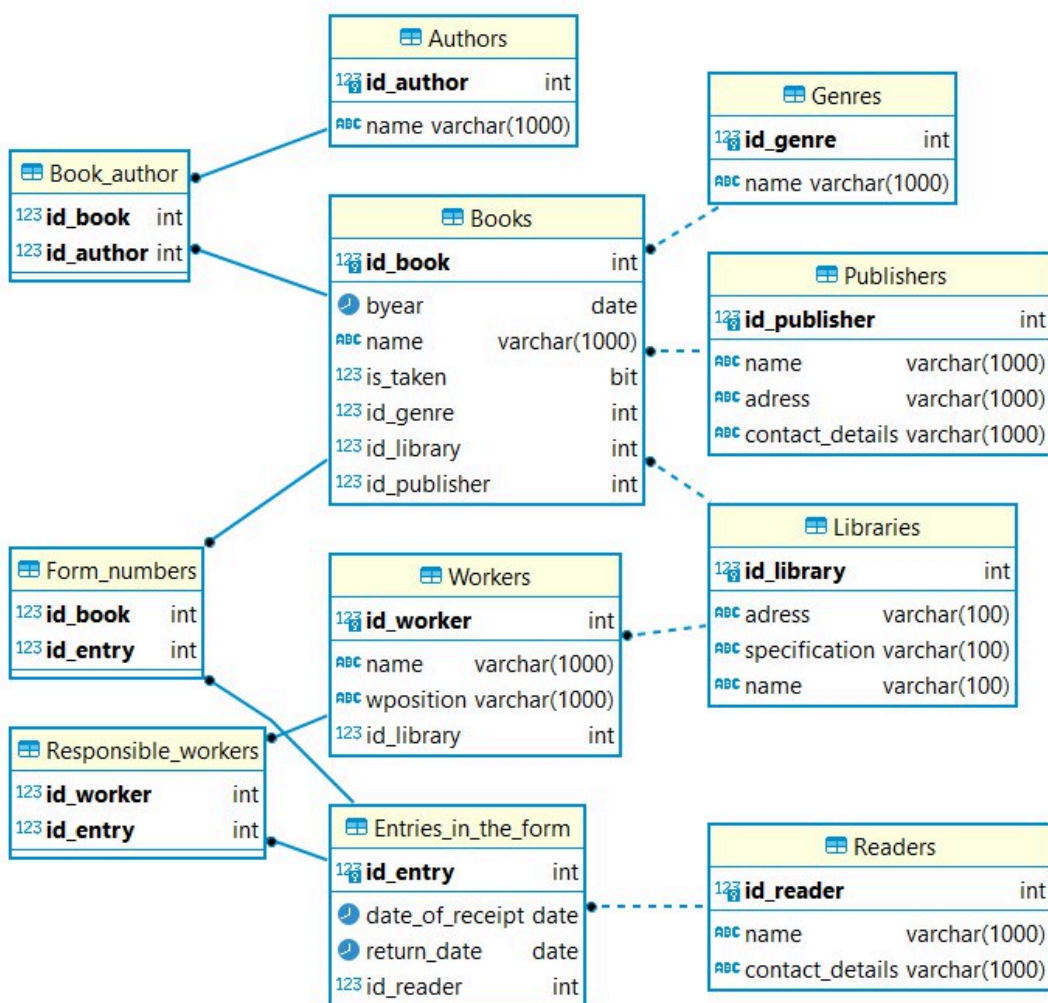


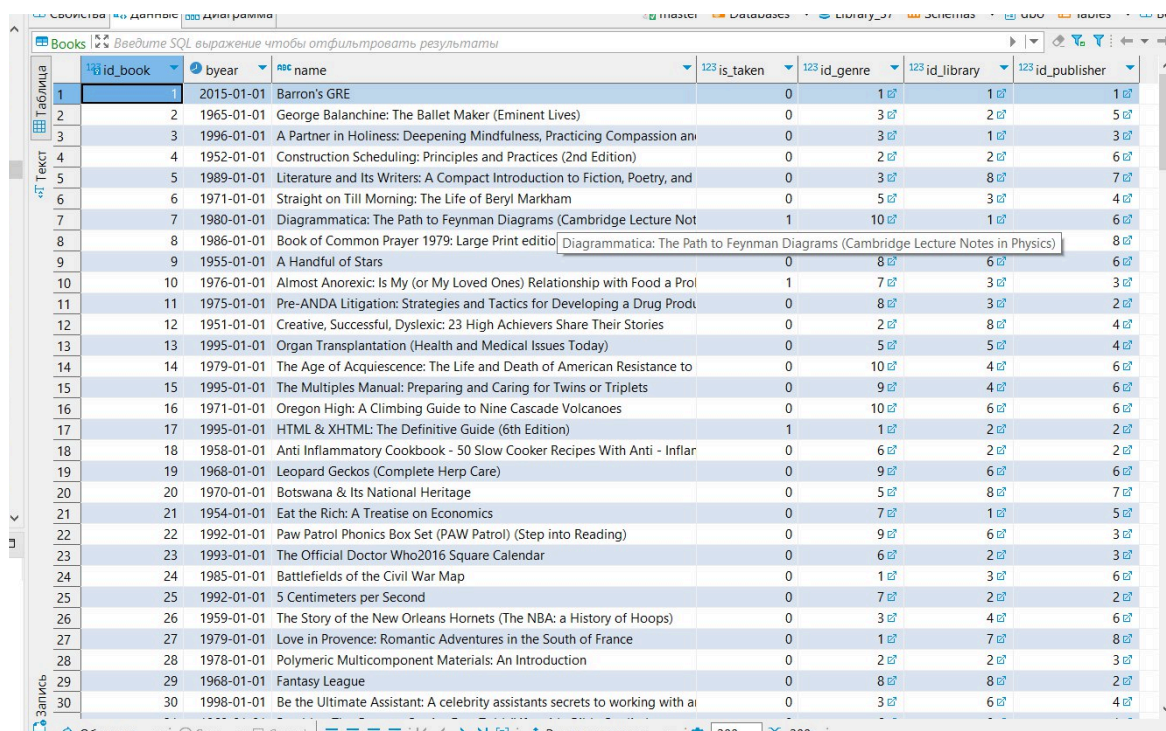
Рисунок 3. Типы данных в модели

3.3. Нормализация базы данных

3 нормальная форма предполагает нахождение в первой и второй, а также отсутствие транзитивных зависимостей. Разработанная база данных находится в 3НФ, поскольку атрибуты атомарны, нет упорядочивания в таблице, нет повторов строк (значит соответствует 1НФ); у каждой таблицы есть первичный ключ, все атрибуты зависимы от него (значит соответствует 2НФ); отсутствует транзитивность.

3.4. Заполнение таблиц данными

Таблицы были заполнены выгрузкой информации из интернета с разработкой кода на Python: особенно заполнялись данные, связанные с книгами (авторы, издательства, названия и т.д.) (Рисунок 4). Информация взята из других библиотек, поскольку в городе Мичуринске отсутствуют хорошие электронные каталоги книг.



	id_book	year	name	is_taken	id_genre	id_library	id_publisher
1	1	2015-01-01	Barron's GRE	0	1	1	1
2	2	1965-01-01	George Balanchine: The Ballet Maker (Eminent Lives)	0	3	2	5
3	3	1996-01-01	A Partner in Holiness: Deepening Mindfulness, Practicing Compassion and	0	3	1	3
4	4	1952-01-01	Construction Scheduling: Principles and Practices (2nd Edition)	0	2	2	6
5	5	1989-01-01	Literature and Its Writers: A Compact Introduction to Fiction, Poetry, and	0	3	8	7
6	6	1971-01-01	Straight on Till Morning: The Life of Beryl Markham	0	5	3	4
7	7	1980-01-01	Diagrammatica: The Path to Feynman Diagrams (Cambridge Lecture Not	1	10	1	6
8	8	1986-01-01	Book of Common Prayer 1979: Large Print edition				8
9	9	1955-01-01	A Handful of Stars	0	8	6	6
10	10	1976-01-01	Almost Anorexic: Is My (or My Loved Ones) Relationship with Food a Prol	1	7	3	3
11	11	1975-01-01	Pre-ANDA Litigation: Strategies and Tactics for Developing a Drug Prodt	0	8	3	2
12	12	1951-01-01	Creative, Successful, Dyslexic: 23 High Achievers Share Their Stories	0	2	8	4
13	13	1995-01-01	Organ Transplantation (Health and Medical Issues Today)	0	5	5	4
14	14	1979-01-01	The Age of Acquiescence: The Life and Death of American Resistance to	0	10	4	6
15	15	1995-01-01	The Multiples Manual: Preparing and Caring for Twins or Triplets	0	9	4	6
16	16	1971-01-01	Oregon High: A Climbing Guide to Nine Cascade Volcanoes	0	10	6	6
17	17	1995-01-01	HTML & XHTML: The Definitive Guide (6th Edition)	1	1	2	2
18	18	1958-01-01	Anti Inflammatory Cookbook - 50 Slow Cooker Recipes With Anti - Infla	0	6	2	2
19	19	1968-01-01	Leopard Geckos (Complete Herp Care)	0	9	6	6
20	20	1970-01-01	Botswana & Its National Heritage	0	5	8	7
21	21	1954-01-01	Eat the Rich: A Treatise on Economics	0	7	1	5
22	22	1992-01-01	Paw Patrol Phonics Box Set (PAW Patrol) (Step into Reading)	0	9	6	3
23	23	1993-01-01	The Official Doctor Who 2016 Square Calendar	0	6	2	3
24	24	1985-01-01	Battlefields of the Civil War Map	0	1	3	6
25	25	1992-01-01	5 Centimeters per Second	0	7	2	2
26	26	1959-01-01	The Story of the New Orleans Hornets (The NBA: a History of Hoops)	0	3	4	6
27	27	1979-01-01	Love in Provence: Romantic Adventures in the South of France	0	1	7	8
28	28	1978-01-01	Polymeric Multicomponent Materials: An Introduction	0	2	2	3
29	29	1968-01-01	Fantasy League	0	8	8	2
30	30	1998-01-01	Be the Ultimate Assistant: A celebrity assistants secrets to working with ai	0	3	6	4

Рисунок 4. Заполненные данные в таблице books

4. Разработка

4.1. Настройка индексов

Индексы помогают ускорить выполнение сложных и даже простых запросов в базе данных. Они включают условия фильтрации, которые оптимизируют работу запросов, в которых они используются.

Для примера был использован 3 запрос из раздела запросов с агрегированием и выражением JOIN, включающим не менее 2-х таблиц:

- Время выполнения до использования индексов было равно 96 ms (Рисунок 5);
- Время выполнения после использования индексов сократилось почти ровно в 2 раза, свидетельствуя о том, что индексы ускоряют работу запросов (Рисунок 6).

```
L.name AS library_name,  
COUNT(*) AS worker_count  
FROM  
  Libraries L  
INNER JOIN Workers W ON L.id_library = W.id_library  
GROUP BY  
  L.name;  
--3  
SELECT  
  A.name AS author_name,  
  COUNT(DISTINCT L.id_library) AS library_count  
FROM  
  Authors A  
INNER JOIN Book_author BA ON A.id_author = BA.id_author  
INNER JOIN Books B ON BA.id_book = B.id_book  
INNER JOIN Libraries L ON B.id_library = L.id_library  
GROUP BY  
  A.name;
```

--Запросы с EXISTS
SFI FCT

ульгат 1 X

ECT A.name AS author_name, COUNT(DISTINCT L.id_library) AS library_count

author_name	library_count
A. C. Atkinson	1
A. Dan Tarlock	1
A. J. McClane	1
A.P. Jones	1
Abby Klein	1
Abu Hamid Muhammad al-Ghazali	1
Ace McCloud	1
Adam Rex	1
Adam Smith	1
Adamari Lopez	1
Adeline Yen Mah	1
Adyashanti	1

Обновить Save Cancel Экспорт данных ... 200 200+ 200 строк получено - 96ms

Рисунок 5. Время до использования индексов

```

L.name AS library_name,
COUNT(*) AS worker_count
FROM
  Libraries L
INNER JOIN Workers W ON L.id_library = W.id_library
GROUP BY
  L.name;
--3
SELECT
  A.name AS author_name,
  COUNT(DISTINCT L.id_library) AS library_count
FROM
  Authors A
INNER JOIN Book_author BA ON A.id_author = BA.id_author
INNER JOIN Books B ON BA.id_book = B.id_book
INNER JOIN Libraries L ON B.id_library = L.id_library
GROUP BY
  A.name;
--Запросы с EXISTS
SELECT

```

Результат 1

SELECT A.name AS author_name, COUNT(DISTINCT L.id_library) AS library_count

	author_name	library_count
1	A. C. Atkinson	1
2	A. Dan Tarlock	1
3	A. J. McClane	1
4	A.P. Jones	1
5	Abby Klein	1
6	Abu Hamid Muhammad al-Ghazali	1
7	Ace McCloud	1
8	Adam Rex	1
9	Adam Smith	1
10	Adamari Lopez	1
11	Adeline Yen Mah	1
12	Adyashanti	1
13	Ahmed Rashid	1

Обновить Save Cancel Экспорт данных ... 200 200+ 200 строк получено - 56ms

Рисунок 6. Время после применения индексов

4.2. Запросы

1. Простой запрос с условием и формулами в SELECT (первый)

Выводит самый частый жанр книг в базе данных.

```

SELECT TOP 1 id_genre, COUNT(*) AS genre_count
FROM Books
WHERE byear > '2010-01-01'
GROUP BY id_genre
ORDER BY genre_count DESC

```

2. Простой запрос с условием и формулами в SELECT (второй)

Выводит всех сотрудников одной библиотеки с id = 1.

```
SELECT w.id_worker, w.name, w.wposition  
FROM Workers w  
WHERE w.id_library = 1
```

3. Запрос с коррелированным подзапросом в SELECT (первый)

Запрос выводит названия книг и названия издательств, которые были взяты из библиотек. Это может быть актуально, когда создается список взятых книг. Обычно библиотекари собирают эти данные вручную из формуляров.

```
SELECT b.name as [Book name], (SELECT p.name FROM Publishers p  
WHERE b.id_publisher = p.id_publisher) as [Publisher name]  
FROM Books b  
WHERE b.is_taken = 1
```

4. Запрос с коррелированным подзапросом в SELECT (второй)

Запрос выводит имена рабочих, которые работают на должности охранника в библиотеке с id = 1, и адреса библиотек, где они работают. Аналогично может быть актуально для других должностей.

```
SELECT w.name as [worker name], (SELECT l.adress FROM Libraries l  
WHERE w.id_library = l.id_library) as [library adress]  
FROM Workers w  
WHERE w.wposition = 'Guard' AND w.id_library = 1
```

5. Запрос с подзапросом в FROM (первый)

Запрос выводит названия книг, которые есть только в библиотеках категории общие (general) и которые не были взяты. Фактически выводится каталог книг обычных библиотек, чтобы читатели могли посмотреть какие книги доступны в обычных библиотеках.

```
SELECT b.name as [book name]  
FROM Books b
```

```

INNER JOIN (SELECT l.id_library, l.specification FROM Libraries l
WHERE l.specification = 'General') as Lib
on b.id_library = Lib.id_library
WHERE b.is_taken = 0

```

6. Запрос с подзапросом в FROM (второй)

Запрос позволяет получить имена читателей, которые взяли книги до 01-06-2023, и соответствующие названия этих книг. Таким образом, можно вывести список читателей и книг, которые сейчас “законно” и без просрочек взяты людьми.

```

SELECT r.name AS reader_name, b.name AS book_name
FROM Readers AS r
JOIN (
SELECT id_reader, id_book
FROM Entries_in_the_form AS eitf
JOIN Form_numbers AS fn ON eitf.id_entry = fn.id_entry
WHERE eitf.return_date > '2023-06-01'
) AS fn ON r.id_reader = fn.id_reader
JOIN Books AS b ON fn.id_book = b.id_book

```

7. Запрос с подзапросом в FROM, агрегированием, группировкой и сортировкой (первый)

Выводит адреса библиотек и количество книг в них в зависимости от спецификации (в нашем случае general). Так можно оценить количество изданий в книжном фонде.

```

SELECT l.adress, l.specification, COUNT(*) AS book_count
FROM (
SELECT id_library, adress, specification
FROM Libraries
WHERE specification = 'General'
) AS l
JOIN Books AS b ON l.id_library = b.id_library
GROUP BY l.adress, l.specification

```

ORDER BY book_count DESC

8. Запрос с подзапросом в FROM, агрегированием, группировкой и сортировкой (второй)

Запрос для получения информации о количестве книг, написанных каждым автором, и отсортированных по количеству книг в убывающем порядке. Это позволяет оценить какие авторы достаточно представлены в библиотеке, а каких авторов требуется добавить в библиотеки города (особенно это касается школьной литературы).

```
SELECT A.name AS author_name, COUNT(*) AS book_count
FROM (SELECT BA.id_author, B.name
FROM Book_author BA
INNER JOIN Books B ON BA.id_book = B.id_book
) AS Subquery
INNER JOIN Authors A ON Subquery.id_author = A.id_author
GROUP BY A.name
ORDER BY book_count DESC
```

9. Запрос с коррелированным подзапросом в WHERE (первый)

Запрос для получения информации о книгах, которые были взяты читателями и не возвращены после указанной даты. Так можно получать список книг, которые не возвращены до сих пор и выявлять должников.

```
SELECT
B.name AS book_name,
E.return_date AS return_date
FROM
Books B
INNER JOIN Form_numbers FN ON B.id_book = FN.id_book
INNER JOIN Entries_in_the_form E ON FN.id_entry = E.id_entry
INNER JOIN Readers R ON R.id_reader = E.id_reader
WHERE
```

```

B.is_taken = 1
AND E.return_date < '2023-06-01'
AND R.id_reader IN (
SELECT E2.id_reader
FROM Entries_in_the_form E2
WHERE E2.id_entry = E.id_entry);

```

10. Запрос с коррелированным подзапросом в WHERE (второй)

Запрос для нахождения книг в конкретном жанре (Fiction) с учетом что книга доступна для читателей.

```

SELECT b.name
FROM Books b
WHERE b.is_taken = 0
AND b.id_genre = (
SELECT g.id_genre
FROM Genres g
WHERE g.name = 'Fiction')

```

11. Запрос, использующий оконную функцию LAG или LEAD для выполнения сравнения данных в разных периодах

Запрос сравнивает текущую дату возврата с предыдущей и следующей датами у книг.

```

SELECT id_entry, date_of_receipt, return_date,
LAG(return_date) OVER (ORDER BY date_of_receipt) AS
previous_return_date,
LEAD(return_date) OVER (ORDER BY date_of_receipt) AS
next_return_date
FROM Entries_in_the_form

```

12. Запрос с агрегированием выражением JOIN, включающим не менее 2 таблиц (первый)

Запрос подсчитывает количество книг автора по жанрам. Актуально для аналитиков и читателей, чтобы оценивать какой жанр более привычен писателю.

```
SELECT
G.name AS genre_name,
A.name AS author_name,
COUNT(B.id_book) AS total_books
FROM
Books B
JOIN Genres G ON B.id_genre = G.id_genre
JOIN Authors A ON B.id_book = A.id_author
JOIN Book_author ba ON B.id_book = ba.id_book
GROUP BY
G.name, A.name
```

13. Запрос с агрегированием и выражением JOIN, включающим не менее 2 таблиц (второй)

Запрос подсчитывает количество работников в каждой библиотеке.

```
SELECT L.name AS library_name, COUNT(*) AS worker_count
FROM Libraries L
INNER JOIN Workers W ON L.id_library = W.id_library
GROUP BY L.name
```

14. Запрос с агрегированием и выражением JOIN, включающим не менее 2 таблиц (третий)

Запрос, который подсчитывает в скольких библиотеках есть книги каждого автора.

```
SELECT A.name AS author_name, COUNT(DISTINCT L.id_library)
AS library_count
FROM Authors A
INNER JOIN Book_author BA ON A.id_author = BA.id_author
INNER JOIN Books B ON BA.id_book = B.id_book
```

```
INNER JOIN Libraries L ON B.id_library = L.id_library  
GROUP BY A.name
```

15. Запрос с EXISTS

Запрос, который находит книги, которые никогда не берут читатели (т.е. про них нет записей в формуляре). Позволяет найти книги, которые не интересны читателям, и соответственно библиотекари дальше принимают решение либо актуализировать внимание посетителей или избавиться от книги.

```
SELECT  
B.name AS book_name,  
A.name AS author_name  
FROM  
Books B  
INNER JOIN Book_author BA ON B.id_book = BA.id_book  
INNER JOIN Authors A ON BA.id_author = A.id_author  
WHERE  
NOT EXISTS (  
SELECT *  
FROM Form_numbers fn  
WHERE fn.id_book = B.id_book)
```

16. Запрос, использующий манипуляции с множествами

Запрос выводит имена авторов, чьи книги были взяты из библиотеки, и книги относятся к конкретному жанру. Библиотекарям приходится ежемесячно составлять полки с книгами, которые не так часто берут в библиотеке, чтобы привлечь внимание читателей. Однако каждый месяц обычно является тематическим и выбирается один конкретный жанр. Таким образом, можно посмотреть книги какого автора в конкретном жанре на данный момент никто не брал. Из этой выборки можно быстрее составлять подборки библиотекарям.

```
SELECT a.name  
FROM Authors a
```

```

WHERE a.id_author IN (SELECT ba.id_author
FROM Book_author ba
WHERE ba.id_book IN (SELECT b.id_book
FROM Books b
WHERE b.is_taken = 0))
INTERSECT
SELECT a2.name
FROM Authors a2
WHERE a2.id_author IN (SELECT ba2.id_author
FROM Book_author ba2
WHERE ba2.id_book IN (SELECT b2.id_book
FROM Books b2
WHERE b2.id_genre = 1))

```

17. Запрос с внешним соединением и проверкой на наличие NULL

В таблице Publishers могут быть пропуски, данный запрос выводит выборку книг, у которых нет информации об издателе. Это актуально, чтобы дорабатывать базу данных и случайные пропуски в ней и отслеживать количество книг, у которых невозможно найти издательский дом.

```

SELECT Books.name, Publishers.name AS publisher
FROM Books
LEFT JOIN Publishers ON Books.id_publisher = Publishers.id_publisher
WHERE Publishers.name IS NULL;

```

18. Запрос с агрегированием и выражением JOIN, включающим не менее 3 таблиц/выражений

Запрос позволяет получить информацию о том, сколько раз бралась та или иная книга. Поможет библиотекарям выявить износ книги и поможет аналитикам посмотреть какие книги актуальны и насколько.

```

SELECT B.name AS book_name, G.name AS genre_name,
COUNT(*) AS total_entries
FROM Books B

```

```

JOIN Genres G ON B.id_genre = G.id_genre
JOIN Form_numbers FN ON B.id_book = FN.id_book
JOIN Entries_in_the_form E ON FN.id_entry = E.id_entry
GROUP BY B.name, G.name

```

19. Запрос с CASE (IIF) и агрегированием

Запрос составляет таблицу с состоянием книг согласно записям из формуляров: если книга не возвращена на момент сегодня, а срок возвращения пропущен (т.е. return_date меньше сегодняшней даты и состояние книги “взята”), то читатель опаздывает и надо напомнить ему о задолженной книге; если книга взята, но срок сдачи еще не подошел, то статус, что книга взята, а если статус книги “не взята”, то книга возвращена

```

SELECT B.name AS book_name,
COUNT(*) AS total_entries,
SUM(CASE WHEN E.return_date >= '2023-06-01' AND B.is_taken = 1
THEN 1 ELSE 0 END) AS later_returns,
SUM(CASE WHEN E.return_date < '2023-06-01' AND B.is_taken = 1
THEN 1 ELSE 0 END) AS earlier_returns,
SUM(CASE WHEN E.return_date > GETDATE() AND B.is_taken = 1
THEN 1 ELSE 0 END) AS late,
CASE
WHEN E.date_of_receipt >= '2023-06-01' THEN 'Reader is reading'
WHEN E.return_date < '2023-06-01' THEN 'Book returned'
WHEN E.return_date > GETDATE() THEN 'Reader is late'
ELSE NULL
END AS return_status
FROM Books B
JOIN Form_numbers FN ON B.id_book = FN.id_book
JOIN Entries_in_the_form E ON FN.id_entry = E.id_entry
GROUP BY B.name, E.return_date, E.date_of_receipt

```

20. Запрос с HAVING и агрегированием

Выводит всех авторов определенного жанра, у кого книг меньше, чем 3, и количество их книг. Это важно, чтобы отследить представленность писателей в библиотеке. Однако для авторов некоторых жанров, например, авторов научных статей, проблема маленького количества книг не критична (обычно они представлены по 1 книге), а для писателей-классиков наличие книг менее чем в 3х экземплярах может быть проблемой, поэтому сделана выборка по жанрам. Этот список сразу поможет понять книги каких писателей важно докупить в книжный фонд.

```
SELECT ba.id_author, b.id_genre, COUNT(*) as book_count
FROM Book_author ba
JOIN Books b on b.id_book = ba.id_book
WHERE b.id_genre = 1
GROUP BY id_author, id_genre
HAVING COUNT(*) <= 3
```

21. Запрос SELECT INTO для подготовки выгрузки

После выполнения этого запроса будет создана новая таблица "ExportedBooks", которая содержит выбранные столбцы и данные из существующих таблиц. Эту таблицу можно использовать для дальнейшей обработки или выгрузки данных.

```
SELECT B.id_book, B.name, A.name AS author_name, G.name AS
genre_name, P.name AS publisher_name
INTO ExportedBooks
FROM Books B
JOIN Book_author BA ON B.id_book = BA.id_book
JOIN Authors A ON BA.id_author = A.id_author
JOIN Genres G ON B.id_genre = G.id_genre
JOIN Publishers P ON B.id_publisher = P.id_publisher
```

4.3. Хранимые процедуры

1. Процедуры, создающие данные в таблицы Workers, Readers, Books и Book_author:

- Добавление нового сотрудника

```
CREATE PROCEDURE Add_Worker (  
    @name VARCHAR(1000),  
    @wposition VARCHAR(1000),  
    @id_library INT)  
AS  
BEGIN  
    INSERT INTO Workers (name, wposition, id_library)  
    VALUES (@name, @wposition, @id_library)  
END
```

- Добавление новой книги

```
CREATE PROCEDURE Add_Reader (  
    @name VARCHAR(1000),  
    @contact_details VARCHAR(1000))  
AS  
BEGIN  
    INSERT INTO Readers (name, contact_details)  
    VALUES (@name, @contact_details)  
END
```

- Добавление новой книги

```
CREATE PROCEDURE Add_Book (  
    @byear DATE,  
    @name VARCHAR(1000),  
    @id_genre INT,  
    @id_library INT,  
    @id_publisher INT)  
AS  
BEGIN  
    INSERT INTO Books (byear, name, is_taken, id_genre, id_library,  
    id_publisher)  
    VALUES (@byear, @name, 0, @id_genre, @id_library,  
    @id_publisher)
```

END

- Объединение автора и книги в сводной таблице

```
CREATE PROCEDURE Add_Author_Book (  
    @id_author INT,  
    @id_book INT)  
AS  
BEGIN  
    INSERT INTO Book_author (id_author, id_book)  
    VALUES (@id_author, @id_book)  
END
```

2. Процедура для оценки количества взятых книг в библиотеках города. (Чтобы не создавать нехватки книг или отражать проблемы в случае большого спроса на книги в библиотеке). В результате показываются значения “нет взятых книг”, “мало взятых книг” или “много взятых книг” на основе количества взятых книг по отношению к общему количеству книг в конкретной библиотеке.

```
CREATE PROCEDURE library_book_situation (@id_library INT)  
AS  
BEGIN  
    DECLARE @bookCount INT  
    SELECT @bookCount = COUNT(*)  
    FROM Books b join Libraries l on b.id_library = l.id_library  
    WHERE b.is_taken = 1 and l.id_library = @id_library  
    IF @bookCount = 0  
    BEGIN  
        SELECT 'Нет взятых книг' AS Result  
    END  
    ELSE IF @bookCount < (SELECT COUNT(*) FROM Books b join  
Libraries l on      b.id_library = l.id_library WHERE l.id_library =  
@id_library) / 2  
    BEGIN  
        SELECT 'Мало взятых книг' AS Result  
    END
```

```

ELSE
BEGIN
    SELECT 'Много взятых книг' AS Result
END
END

```

3. Процедура с try catch и transaction

При добавлении записи в entries in the form добавляет запись в form number и меняет в таблице book значение is_taken с 0 на 1, но если книга уже взята (стоит 1), то выходит ошибка, что книга уже взята.

```

CREATE PROCEDURE get_the_book_and_fill_the_entry (@id_book
INT, @return_date DATE, @id_reader INT)
AS
BEGIN
BEGIN TRY
BEGIN TRANSACTION
DECLARE @isTaken INT
SELECT @isTaken = is_taken
FROM Books
WHERE id_book = @id_book
IF @isTaken = 1
BEGIN
DECLARE @errorMessage NVARCHAR(500)
SET @errorMessage = 'Ошибка: Книга уже взята'
RAISERROR (@errorMessage, 16, 1)
END
UPDATE Books
SET is_taken = 1
WHERE id_book = @id_book
DECLARE @entry_id INT
INSERT INTO Entries_in_the_form (date_of_receipt, return_date,
id_reader)
VALUES (GETDATE(), @return_date, @id_reader)
SET @entry_id = SCOPE_IDENTITY()
INSERT INTO Form_numbers (id_book, id_entry)

```

```
VALUES (@id_book, @entry_id)
COMMIT TRANSACTION
END TRY
BEGIN CATCH
PRINT 'Произошла ошибка: ' + ERROR_MESSAGE()
ROLLBACK TRANSACTION
END CATCH
END
```

4. Обновляет данные адреса и контактные детали в таблице издательства. Это необходимо, потому что некоторые издательства с течением времени могут либо перестать существовать и не иметь никаких контактных данных и адресов, или они могут изменить их.

```
CREATE PROCEDURE Change_adress_contact_publisher (
@id_publisher INT,
@adress VARCHAR(1000),
@contact_details VARCHAR(1000)
)
AS
BEGIN
UPDATE Publishers
SET adress = @adress, contact_details = @contact_details
WHERE id_publisher = @id_publisher
END
```

5. Удаляет данные о читателе, если тот перестал или больше не может пользоваться услугами библиотеки.

```
CREATE PROCEDURE Delete_reader (@id_reader int)
AS
BEGIN
Delete
From Readers
Where id_reader = @id_reader
END
```

4.4. Триггер

Данный триггер автоматически устанавливает дату получения книги в текущую дату при вставке новой записи в таблицу "Entries_in_the_form".

```
CREATE TRIGGER SetReceiptDate
ON Entries_in_the_form
AFTER INSERT
AS
BEGIN
UPDATE Entries_in_the_form
SET date_of_receipt = GETDATE()
WHERE id_entry IN (SELECT id_entry FROM inserted);
END
```

4.5. Функции

1. Данная функция для получения всех доступных книг в определенной библиотеке в виде таблицы с названием, годом выпуска. Фактически формирует каталог доступных книг библиотеки.

```
CREATE FUNCTION Get_Available_Books(@id_library INT)
RETURNS TABLE
AS
RETURN
(SELECT b.id_book, b.byear, b.name, b.id_library
FROM Books b
JOIN Libraries l ON b.id_library = l.id_library
WHERE b.is_taken = 0
AND b.id_library = @id_library)
```

2. Данная функция помогает посчитать количество экземпляров по названию книги и имени автора. Это может быть очень важно, например, с книгами из школьной литературы: посмотреть сколько экземпляров книги “Преступление и наказание” Достоевского в городе, чтобы понять, хватает ли нынешним школьникам

экземпляров. Также можно смотреть есть ли запасные экземпляры искомой книги и т.д.

```
CREATE FUNCTION Get_num_of_Books_by_Author_Name(  
    @authorName VARCHAR(1000),  
    @bookName VARCHAR(1000)  
)  
RETURNS INT  
AS  
BEGIN  
    DECLARE @bookCount INT  
    SET @bookCount = (  
        SELECT COUNT(*)  
        FROM Books b  
        JOIN Book_author ba ON b.id_book = ba.id_book  
        JOIN Authors a ON ba.id_author = a.id_author  
        WHERE a.name = @authorName AND b.name = @bookName  
    )  
    RETURN @bookCount  
END
```

4.6. Представления

1. Выводит все книги с названиями всех жанров и авторов, которые доступны сейчас в библиотеках (то есть могут быть взяты в библиотеке сейчас). Поможет не делать запрос, а сразу получать данные в виде таблицы.

```
CREATE VIEW Available_books AS  
SELECT B.id_book, B.name AS book_name, B.byear AS Book_year,  
G.name AS genre_name, A.name AS author_name  
FROM Books B  
JOIN Genres G ON B.id_genre = G.id_genre  
JOIN Book_author BA ON B.id_book = BA.id_book  
JOIN Authors A ON BA.id_author = A.id_author  
WHERE is_taken = 0
```

2. Показывает сколько записей в формуляре за сегодняшний день сделал каждый сотрудник. Это может быть способом проанализировать качество работы или отследить тенденцию среди сотрудников. Также эти данные могут использоваться для премирования сотрудников начальством.

```
CREATE VIEW Workload_of_employees AS
SELECT  W.name  AS  worker_name,  COUNT(E.id_entry)  AS
total_entries
FROM Workers W
JOIN Responsible_workers RW ON W.id_worker = RW.id_worker
JOIN Entries_in_the_form E ON RW.id_entry = E.id_entry
WHERE E.date_of_receipt = GETDATE()
GROUP BY W.name
```

5. Отчёты

5.1. Отчёт в Tableau

В данном отчёте можно увидеть распределение по жанрам во всех библиотеках города (Рисунок 7). Благодаря нему можно понять какие жанры представлены в библиотеке, в каких библиотеках равномерно или неравномерно распределились жанры.

По этим данным можно сделать корректные выводы о состоянии книжного фонда. Например, по этим данным видно, что в библиотеке №3 плохо представлен жанр Fiction, хотя он является самым популярным среди всех. Значит необходимо сделать закупку книг в данном жанре в библиотеку №3.

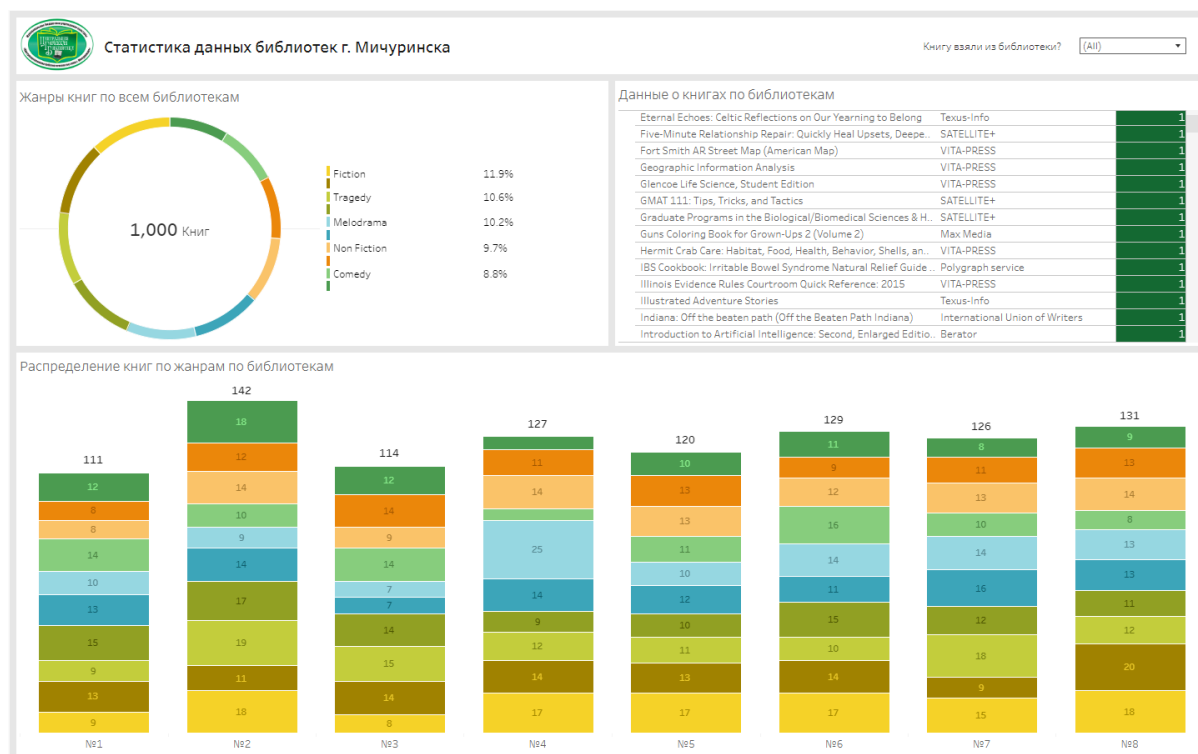


Рисунок 7. Отчет в Tableau о предпочтениях в жанрах по всем книгам

5.2. Отчёт в Tableau

В данном отчёте представлено распределение книг, которые доступны в библиотеке и те, которые сейчас находятся у читателей (Рисунок 8-9). Так можно отследить тренды на литературу среди

посетителей в данный момент и понять какие жанры не воспользовались спросом в месяце.

Например, согласно отчёту по доступным книгам, жанр трагедий не пользовался успехом в данном месяце, поэтому библиотекарям есть смысл создавать полки с подборками книг в жанре Tragedy.

По отчёту о взятых книгах видно, что в этом месяце предпочитали романы и драмы. Это может быть связано с выходом новой книги, сезоном или каким-то социальным явлением. Также можно проследить, что в библиотеке №3 и №7 книг взяли мало, стоит оценить качество работы сотрудников в этом месяце.

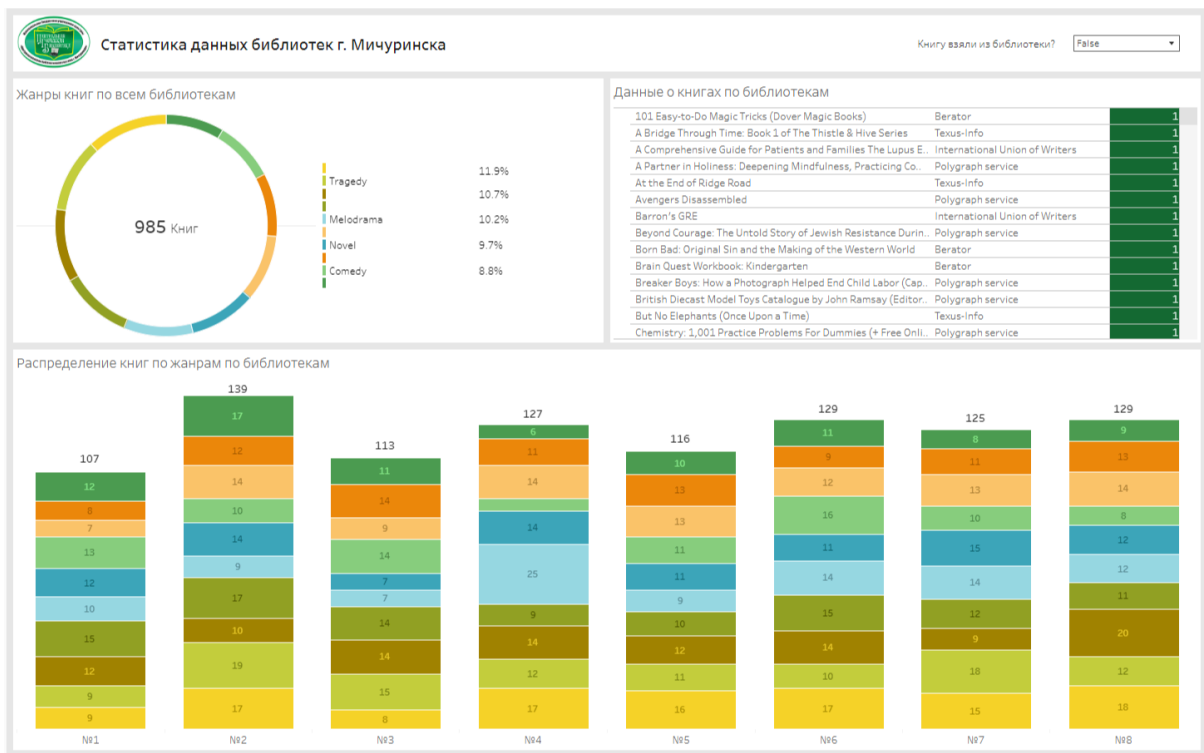


Рисунок 8. Отчет в Tableau о предпочтениях в жанрах по книгам, которые сейчас доступны в библиотеке

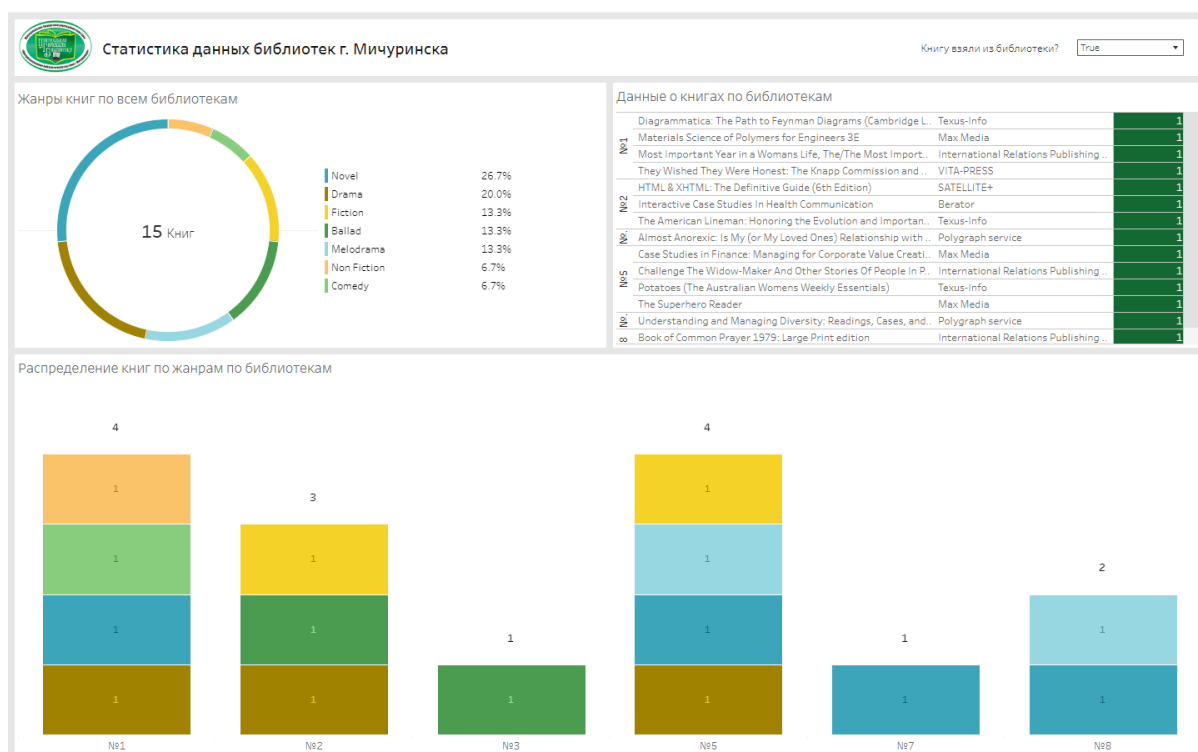


Рисунок 9. Отчет в Tableau о предпочтениях в жанрах по книгам, которые взяли читатели

5.3. Отчёт в Excel

На данном графике можно увидеть и оценить старость книжного фонда. Стоит рассмотреть какие книги на данном этапе слишком старые и их количество. Если их много, то стоит провести ревизию и закупить новые экземпляры: книги изнашиваются, устаревают данные и находятся ошибки. Новые книги тоже стоит рассматривать: если слишком молодые книги будут в большом количестве, то стоит провести ревизию и проверить интерес у читателей к ним. В современности печатается большое количество невостребованных книг, если новые книги окажутся неинтересными читателям, то их стоит убрать из книжного фонда.

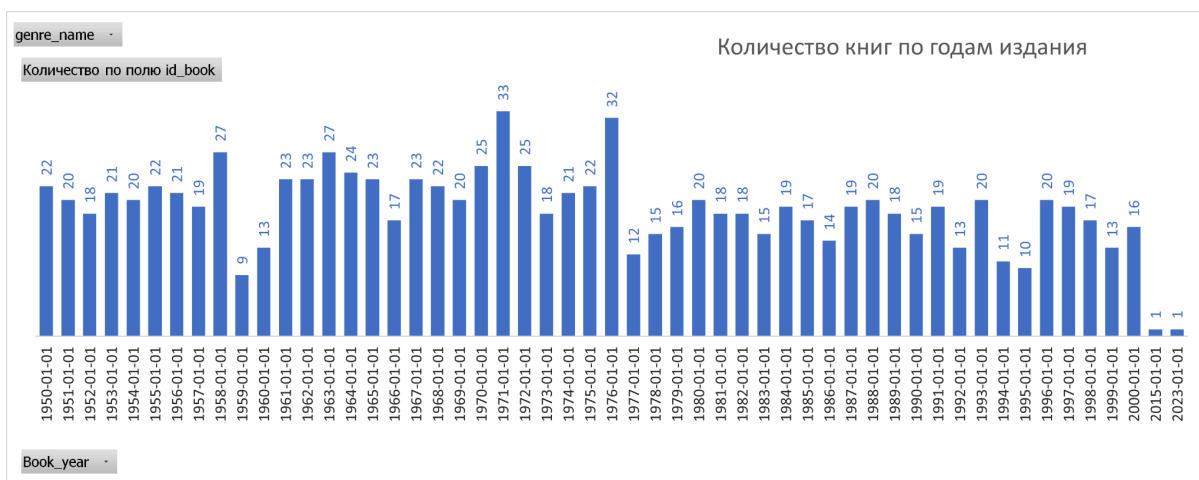


Рисунок 9. График “возраста” книжного фонда библиотек